Google Summer of Code, 2026

Proposal for Checkstyle



Project: Patch Suppression Improvement

Name: Miriyala Jayanth

GitHub: MiryalaJayanth-19

LinkedIn:M. Jayanth

Email: mrufo2k5@gmail.com

Time Zone: Indian Standard Time (IST), UTC +5:30

Project Size: Large

Mentors: Roman Ivanov, Ruslan Diachenko

Table Of Contents

ABOUT ME.....3

PROJECT DETAILS..... 4

UNDERSTANDING THE PROBLEM……..…5

WORKING OF PATCH_FILTER……..6

INTEGRATION WITH CHECKSTYLE……...7

PROPOSED IMPROVEMENTS……9

TESTING……..18

NEW FILTER IN CHECKSTYLE……..19

ONBOARDING ECLIPSE CS……20

PROJECT TIMELINE……..21

EXPECTATIONS FROM MENTORS……23

MY CONTRIBUTIONS……23

WHY ME?.....24

FUTURE WORK……..24

ABOUT ME

Greetings! I'm Miriyala Jayanth, currently pursuing my third year in B-Tech Computer Engineering at Sreenidhi Institute of Science and Technology, Telangana, India. Over the past year, I've immersed myself in learning java , data structures and algorithms,OOPs concepts . I also have some basic knowledge on machine learning and robotics .My strong desire to explore and continuously learn motivates me to constantly discover new fields and technologies.

Open source this word is very new to me, I found it during Hacktober fest and learned about this . I understood that most software technology is open to be used by any person. This made me very interested in the OS world. Also for any fresher entering a software career, this is the best way to gain experience. I made some ui/ux , doc updates, language translations as my beginning contributions to get a complete picture of open source .

I started contributing to checkstyle in the beginning of March. Contributing to checkstyle has made me understand the complexity of engineering , how a single change in line can cause a huge cascade effect in the software. This changed me to think how a simple decision could shape the life of one.

I enjoy contributing to Checkstyle because it is a beginner friendly organization, well documented and is very active. Maintainers are always ready to help the contributors with their issues. They are open to discussions on new ideas and implementation .

PROJECT DETAILS

Introducing CheckStyle to new java projects is a challenging task , especially when the project has a large amount of code and is very active in development , and there are no resources to start a new process of code cleanup. It would require extensive efforts and time to be spent on it ,especially when there is legacy code written by previous contributors that becomes a monotonous job which everyone avoids .
The patch-filters project currently exists as a proof-of-concept that requires significant manual setup and has several critical bugs that prevents real-world adoption. The goal of this project is to make it production-ready component that any Java project can adopt It in their own standard CI pipelines.
We need to focus on parsing patch files to get a more precise location of changes and be able to skip violations if the fix for it goes outside of changed lines. For example: user changing line wrapping of long signature of method and we should not demand decreasing the amount of parameters or fixing names, as this will trigger changes in other parts of code. The implementation of this will be discussed in the next sections .

Project Deliverables:

- New filter in CheckStyle that can be applied in the current checkstyles version codebase.
- Handle code logics where required .To give more clear violations for neverSuppressedChecks.Tackling issue #335
- Apply the filter on eclipse-cs to work on each update (address the feedback)
- Single line maven command ,easy to integrate into the CI
- Documentation on usage of filter for better understandment of user
- Making CheckStyle adoption easy.

Understanding the Problem

The Legacy Codebase Problem

Introducing Checkstyle into a large, active project is extremely challenging. When run on a codebase like Guava or HBase for the first time, Checkstylecode written by previous contributors. Nobody wants to fix thousands of violations — almost all from legacy violations written by people who no longer work on the project. Good practice from OpenJDK explicitly discourages changing code without a functional reason. The result: Checkstyle never gets adopted.

The ideal solution is: enforce rules only on lines that were actually written or changed in the current commit or PR. Legacy code stays as-is. New and modified code must comply. This is exactly what the patch-filters project was built to achieve.

The Cascade Effect

As Roman Ivanov described in issue #6689, even when a filter exists, imprecise line-range tracking creates an 'avalanche' problem. When a developer reformats a method signature across multiple lines, the filter marks a wide range of surrounding lines as changed — including the method declaration. Violations on ParameterNumber or MethodLength then fire on the method declaration even though the developer never intended to change the method structure. Fixing those violations triggers changes in callers, which trigger more violations. The cascade makes the tool unusable.

The Multi-Commit PR Gap — Issue #351

Issue #351 reveals a second critical gap: the default patch generation command (git diff HEAD~1 HEAD) only covers the LAST commit. A developer can split violations across multiple commits and submit a clean final commit, hiding all previous violations from the filter. Except for Travis CI - GitHub Actions, Jenkins, and CircleCI do not do this. Eclipse-CS already uses this filter in production (commit 2cb3fb24) but with this same limitation.

FILTER WORKING

The current patch-filter uses CheckStyle version -10.4 and jdk -17 .

- SuppressionPatchFilter & SuppressionJavaPatchFilter are AutomaticBean instances — Checkstyle injects properties from user's XML config and calls finishLocalSetup() to trigger loadPatchFile()
- CrFilterInputStream wraps the patch file input stream and strips all \r characters before JGit sees the content, ensuring Windows-generated patches parse correctly on all platforms
- JGit's Patch class converts simple diff text into FileHeader and HunkHeader objects which LoadPatchFileUtils uses to extract precise changed line ranges per file
- LoadPatchFileUtils builds a filename → List<lineRange> map where each range is [getBeginB(), getEndB()] from JGit's Edit objects representing the new file line positions
- PatchFilterSet holds one SuppressionPatchFilterElement per changed file and start check on the first accepting filter — returning true (show violation) as soon as any element matches
- SuppressionPatchFilterElement.accept() checks filename match first, then routes through isNeverSuppressCheck(), isTreeWalkerChecksMatching(), and isLineMatching() — the TreeWalker bypass at line 69 makes ALL 150+ AST checks pass unconditionally
- JavaPatchFilterElement implements TreeWalkerFilter instead of Filter it gives access to TreeWalkerAuditEvent which carries the full DetailAST — enabling AST traversal that SuppressionPatchFilterElement won’t do
- For checks in CHECK_TO_ANCESTOR_NODES_MAP, getAncestorAst() walks up the AST tree by eventAst.getParent() until it finds the correct closing node type (e.g. CLASS_DEF, METHOD_DEF), then getChildAstLineNo() gets min and max line numbers of all child nodes to check is it mixing with changed ranges

INTEGRATION WITH CHECKSTYLE

The current patch-filter follow the coding style of checkstyle version -10.4 and java 17 , where as the current checkstyle that is in development is 13.4. The coding style is completely different between the two . As the version 13.3 is have more checks added with stricter imposed rules , we ought to get many styling violations . I've run checkstyle current version on patch-filters, It gave 303 violations that need to rectified to integrate into checkstyle.

```
[INFO] ------------------------------------------------------------------------
[INFO] BUILD FAILURE
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  28.479 s
[INFO] Finished at: 2026-03-28T12:37:07+05:30
[INFO] ------------------------------------------------------------------------
[ERROR] Failed to execute goal org.apache.maven.plugins:maven-checkstyle-plugin:3.2.0:check (default-cli) on project pat
ch-filters: You have 303 Checkstyle violations. -> [Help 1]
[ERROR]
[ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch.
[ERROR] Re-run Maven using the -X switch to enable full debug logging.
[ERROR]
[ERROR] For more information about the errors and possible solutions, please read the following articles:
[ERROR] [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/MojoFailureException
```

VIOLATIONS:

1) IndentationCheck →233 violations
   - Due to basic offset rule of 4 spaces
   - Reformat every file that has given the error

2) RightCurlyCheck → 12 violations
   - Due to not placing '}' on the same line of next part of muti block statement
   - Edit the placement of the brackets. (The PR is open once the dependency is cleared it can be merged)

3)CustomImportOrderCheck → 55 violations

- Checkstyle expects imports to be perfectly ordered alphabetically and grouped properly
- Organise imports in each file lexicographically

4) MissingJavaDocTypeCheck and MissingJavaDocMethodCheck → 22 violations

- These complain about public classes or methods that don't have proper Javadoc comments block above them.
- Add javadoc comments at required areas.

TESTING :

As for software engineering , testing is the most important state . It tells whether the work done is success or not . As already checkstyle has automatic testing with clear error messages the testing can be easily done and debugged.

After the build is success we can move forward with the current issues in the code .

NOTE : I Opened a pr solving the right curly braces errors.

PROPOSED IMPROVEMENTS

While i was exploring the codebase i identified the following bugs that need to be handled , or improved for better optimization of the filter .
These are changes i think that should be made for solving the current issues for the filter to be adopted.

1 - SuppressionPatchFilterElement

Fix isTreeWalkerChecksMatching():
The accept() method calls isTreeWalkerChecksMatching() which returns true for ALL TreeWalker-based checks unconditionally, causing every AST-level violation (ParameterNumber, MethodLength, NeedBraces, etc.) to bypass line-range suppression entirely.

```
private static boolean isTreeWalkerChecksMatching(AuditEvent event) {
    boolean result = false;
    final String sourceName = event.getViolation().getSourceName();
    try {
        final Class clazz = Class.forName(sourceName);
        if (ModuleReflectionUtil.isCheckstyleTreeWalkerCheck(clazz)) {
            result = true;
        }
        return result;
    }
    catch (ClassNotFoundException ex) {
        throw new IllegalStateException("Class " + sourceName + " not found", ex);
    }
}
```

Goal : Remove isTreeWalkerChecksMatching() from the accept() OR rewrite it so TreeWalker checks still go through isLineMatching() like all other checks.Every TreeWalker-based check (ParameterNumber, MethodLength, WhitespaceAround, etc.) will now correctly suppress violations on unchanged lines. This directly fixes the cascade effect Roman Ivanov described in issue #6689

2 - GeneratePatchFile

2.1 : Fix Context line in diffformater

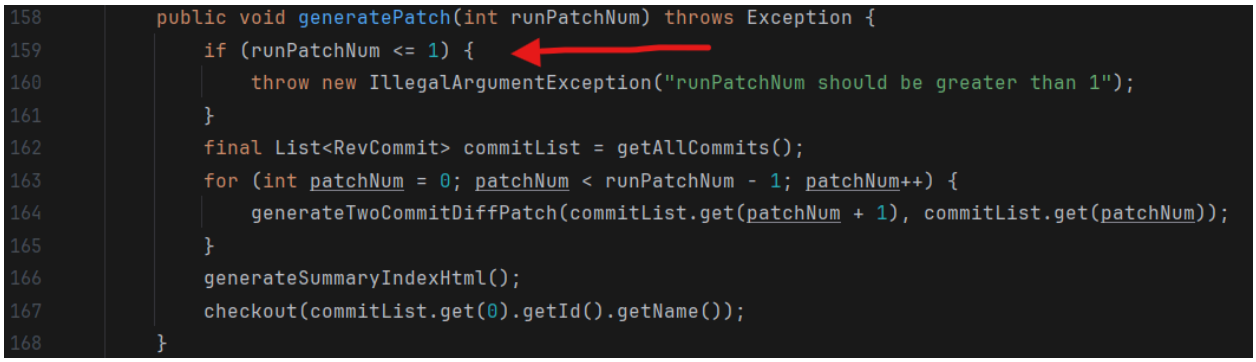
When a file is renamed AND has content changes, the RENAME check causes getLineRangeList() to return an empty list. This means ALL violations in the renamed file are suppressed — even the lines the developer changed alongside the rename.

```
public List<List<Integer>> getLineRangeList() {
 if (!"RENAME".equals(fileHeader.getChangeType().name())) {
        // only processes non-renamed files
        for (HunkHeader hunkHeader : fileHeader.getHunks()) {
        // ... process hunks
        }
 }
 return lineRangeList;  // ← empty for RENAME, all violations suppressed
}
```

Goal : Eliminates the root cause of line-range boundary shift. Proven by GitDiffWithContextSizeDefaultTest vs GitDiffWithContextSizeZeroTest — same patch with context=3 vs context=0 produces Edit(4,4,4,12) vs Edit(3,3,4,12), a shifted start line that causes wrong suppression decisions.

2.2 : Fix runPatchNum Validation - Off-by-One Error

The method throws IllegalArgumentException when runPatchNum <= 1, meaning passing 1 (the most common use case — 'look at my last commit') throws an error instead of working.



```
158     public void generatePatch(int runPatchNum) throws Exception {
159         if (runPatchNum <= 1) {
160             throw new IllegalArgumentException("runPatchNum should be greater than 1");
161         }
162         final List<RevCommit> commitList = getAllCommits();
163         for (int patchNum = 0; patchNum < runPatchNum - 1; patchNum++) {
164             generateTwoCommitDiffPatch(commitList.get(patchNum + 1), commitList.get(patchNum));
165         }
166         generateSummaryIndexHtml();
167         checkout(commitList.get(0).getId().getName());
168     }
```

Goal : Change the condition from <= 1 to < 1 so that passing 1 correctly generates a patch for the last commit.The single most common use case 'validate my last commit' — now works. Previously any developer who passed 1 got an unhelpful exception.

2.3 : Add Whitespace-Ignore Flag to Patch Generation

When a developer wraps a block of code in an if-statement, every inner line gets re-indented. The patch marks all those lines as changed even though no logic changed. As discussed in issue #6689. No -w flag is passed to git diff.



Goal : Add an ignoreWhitespace boolean property to SuppressionPatchFilter/SuppressionJavaPatchFilter. When true, pass -w flag to git diff and set DiffFormatter to use WhitespaceIgnoreChangeFilter.

Prevents indentation-only changes (wrapping code in if/try/for blocks) from triggering violations on every re-indented line. This is the whitespace cascade Roman described and it is one of the most common real-world pain points.

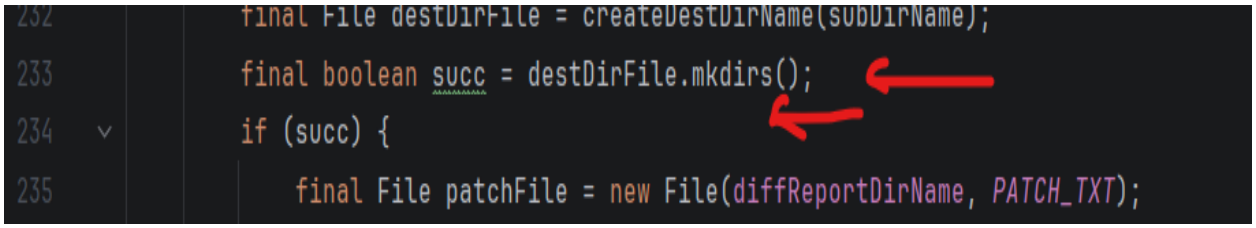
2.4 : Fix Hard-CodeThe base branch passed to diff.groovy is always hard-coded as 'master'. Since GitHub changed its default branch to 'main' in 2020, this silently fails for the majority of modern open source projects.

d master branch.

Goal : Add a baseBranch constructor parameter and use it in the groovy command.

2.5 : Silent failure when directory creation fails

In method genearateTwoCommitDiffPatch() , destDirFile.mkdirs() result is checked but failure is silently swallowed — the entire commit is skipped with no error, no log message, and no exception. Developers would spend their time debugging .



Goal : Throw an IOException with a descriptive message when directory creation fails.Developers will get a clear error instead of wondering why the tool produced no output.

3 - SuppressionPatchFilter/LoadPatchFileUtils.java

3.1 : Handle New Files (/dev/null Origin) properly

New files (where the diff shows /dev/null as source) are treated identically to modified files. They get added to the suppression set with a line range, meaning some violations on brand-new code may be suppressed. Brand new files should have ALL violations shown — there is no legacy excuse.

```
// In loadPatchFile() — no special handling for new
files:
for (FileHeader fileHeader : fileHeaders) {
  final LoadPatchFileUtils utils =
     new LoadPatchFileUtils(fileHeader, strategy);
  // ← no check for ADD change type
  filters.addFilter(element);
}
```

Goal : Check fileHeader.getChangeType(). If it is ADD (new file), do not add to the suppression set so all violations are shown.Brand new files now correctly show all violations.

Previously a developer adding a new file could have some violations silently suppressed, defeating the purpose of the tool.

3.2 : Fix rename file handling - lost content changes

When a file is renamed AND has content changes, the RENAME check causes getLineRangeList() to return an empty list. This means ALL violations in the renamed file are suppressed — even the lines the developer changed alongside the rename.

```
70      public List<List<Integer>> getLineRangeList() {  2 usages  Baratali Izmailov +1
71          final List<List<Integer>> lineRangeList = new ArrayList<>();
72          if (!"RENAME".equals(fileHeader.getChangeType().name())) {
73              for (HunkHeader hunkHeader : fileHeader.getHunks()) {
74                  final EditList edits = hunkHeader.toEditList();
75                  for (Edit edit : edits) {
76                      addSingleLineRange(lineRangeList, edit);
77                  }
78              }
79          }
80          return lineRangeList;
81      }
```

Goal : Process hunks inside RENAME files normally. Only skip processing when the rename has ZERO content changes (no hunks).Developers who rename a file while also making edits will now correctly see violations on the lines they changed. Confirmed by GitDiffWithContextSizeDefaultTest.testHaveRenamedWithNoChangedFile() which shows jgit correctly reports RENAME with 0 hunks for pure renames.

4) Expand Check_to_Ancestor_Nodes_Map for all relevant checks

- **JavaPatchFilterElement.java**

The static map only contains 11 checks (ArrayTrailingComma, AvoidNestedBlocks, CommentsIndentation, DefaultComesLast, DeclarationOrder, EqualsHashCode, FinalLocalVariable, FallThrough, InnerTypeLast, RightCurly, VariableDeclarationUsageDistance).

Checkstyle has 150+ checks. Checks not in the map cannot benefit from the context strategy even when they should.

```
52      private static final Map<String, List<Integer>> CHECK_TO_ANCESTOR_NODES_MAP = new HashMap<>();
53
54      static {
55          CHECK_TO_ANCESTOR_NODES_MAP.put("ArrayTrailingComma",
56                  Arrays.asList(TokenTypes.ARRAY_INIT));
57          CHECK_TO_ANCESTOR_NODES_MAP.put("AvoidNestedBlocks",
58                  Arrays.asList(TokenTypes.SLIST));
59          CHECK_TO_ANCESTOR_NODES_MAP.put("CommentsIndentation",
60                  Arrays.asList(TokenTypes.METHOD_DEF, TokenTypes.SLIST,
61                          TokenTypes.CLASS_DEF, TokenTypes.INTERFACE_DEF,
62                          TokenTypes.RECORD_DEF, TokenTypes.ENUM_DEF));
63          CHECK_TO_ANCESTOR_NODES_MAP.put("DefaultComesLast",
64                  Arrays.asList(TokenTypes.LITERAL_SWITCH));
65          CHECK_TO_ANCESTOR_NODES_MAP.put("DeclarationOrder",
66                  Arrays.asList(TokenTypes.CLASS_DEF, TokenTypes.INTERFACE_DEF,
67                          TokenTypes.ENUM_DEF));
68          CHECK_TO_ANCESTOR_NODES_MAP.put("EqualsHashCode",
69                  Arrays.asList(TokenTypes.CLASS_DEF));
70          CHECK_TO_ANCESTOR_NODES_MAP.put("FinalLocalVariable",
71                  Arrays.asList(TokenTypes.METHOD_DEF,
72                  TokenTypes.VARIABLE_DEF, TokenTypes.CTOR_DEF));
73          CHECK_TO_ANCESTOR_NODES_MAP.put("FallThrough",
74                  Arrays.asList(TokenTypes.LITERAL_SWITCH));
75          CHECK_TO_ANCESTOR_NODES_MAP.put("InnerTypeLast",
76                  Arrays.asList(TokenTypes.CLASS_DEF, TokenTypes.INTERFACE_DEF,
77                          TokenTypes.RECORD_DEF, TokenTypes.ENUM_DEF));
78          CHECK_TO_ANCESTOR_NODES_MAP.put("RightCurly",
```

Goal :Systematically review all Checkstyle checks and add entries for those that fire on structural nodes (class, method, constructor, switch) when the violation is caused by content changes deep inside that structure. ParameterNumber and MethodLength — the two most common cascade-causing checks — will now correctly fire when a developer changes content inside a method, alerting them to structural problems in code they touched. Previously they were either always shown (bypass bug) or never shown (not in map).

5) Documentation

5.1 : Add patchConfig.xml Documentation and TreeWalker Example

The patchConfig.xml only shows the filter with a single file property and no explanation of strategy, optional, or neverSuppressedChecks. It also contains zero TreeWalker checks, meaning the bypass bug (IMPROVEMENT #1) can never be triggered by the reference config.

```
<module
name="com.puppycrawl.tools.checkstyle.filters.SuppressionPatchFil
ter">
  <property name="file" value="${checkstyle.patchfilter.patch}"/>
  <!-- strategy, optional, neverSuppressedChecks not shown -->
</module>
<!-- NO TreeWalker module at all →
```

Goal : Add full property documentation as comments, add a TreeWalker module with ParameterNumber and MethodLength, and add strategy examples. New users reading patchConfig.xml will immediately understand all configuration options. The TreeWalker module addition will also serve as the integration test that validates IMPROVEMENT-#1.

5.2 : Document every fix done

CheckStyle is a very documented software so it clearly validates having descriptive documentation .As the technical changes would confuse the users a comprehensive documentation is needed for every fix that is done .

Goal : Write the comments with full clarity on why the change has been made , what impact does it have .Making the filter to be easily understandable by the user .

6) NEW IMPLEMENTATIONS:

6.1 : Checks under neverSuppressedChecks should give more clear violations

This is open issue #335 , suggested by maintainer : Ruslan Diachenko.

Currently, all violations from the checks under `neverSuppressedChecks` group will be shown, even though the code isn't changed. Investigate checks marked with that property and try to come up with more precise strategies for them. Ideally, if code was changed and that change leads to a new violation that violation should be shown only.



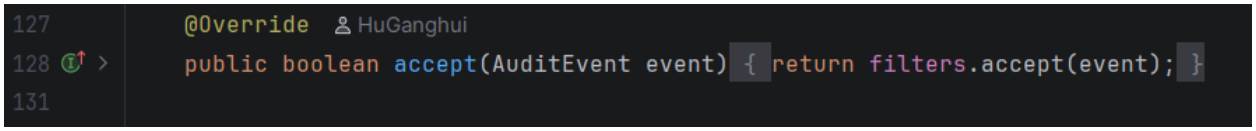
```
146     private boolean isNeverSuppressCheck(AuditEvent event) {  1 usage  HuGanghui
147         boolean result = false;
148         if (neverSuppressedChecks != null) {
149             if (containsShortName(neverSuppressedChecks, event)
150                     || neverSuppressedChecks.contains(event.getModuleId())) {
151                 result = true;
152             }
153         }
154         return result;
155     }
```

Goal : the method isNeverSuppressCheck() in SuppressionPatchFilterElement.java accepts all violations returns true value irrespective of its relation to patch lines.
To address this issue, the implementation will enhance the existing patch filtering logic for checks under the `neverSuppressedChecks` group by making them patch-aware rather than globally enforced. Instead of reporting all violations unconditionally, the filter will be extended to evaluate whether a violation is directly associated with modified code. This will be achieved by mapping each violation to its corresponding line number and enclosing AST node (such as method or class), and then verifying whether it overlaps with the changed regions in the patch. For simple checks, line-based filtering will be applied, while for structural checks, context-aware analysis will ensure that only violations within modified scopes are reported. Additionally, an optional improvement will include comparing violations between the base and patched versions to identify newly introduced issues, ensuring higher precision.

This approach maintains the strictness of `neverSuppressedChecks` while significantly reducing noise by reporting only relevant violations caused by the patch.

6.2 : No diff-based detection at all (Shall be implemented in future)

A major limitation of the current patch-filters approach is the absence of **diff-based violation detection**, which prevents it from distinguishing between pre-existing issues and those newly introduced by a code change. At present, filtering is largely line-based, meaning that any violation occurring on or near modified lines may be reported regardless of whether it already existed before the patch. This leads to significant noise, especially in large or legacy codebases where many violations are already present.





Goal : A more precise solution is to introduce a diff-based mechanism that runs static analysis on both the base version of the code and the patched version, and then computes the difference between the two sets of violations. By identifying only those violations that appear in the patched version but not in the base version, the system can accurately report issues that are truly introduced by the change. This approach ensures high precision, supports incremental adoption of Checkstyle, and aligns well with modern CI/CD practices where developers are expected to fix only newly introduced problems rather than legacy ones.

TESTING

PHASE 1: Unit Testing
Add tests for the following files for changes made :

- SuppressionPatchFilterTest
- JavaPatchFilterElementTest
- LoadPatchUtilsTest
- PatchFilterSetTest
- Adding edge testcase (emptyrepo,longline,singleline..etc)

PHASE 2 : Integration Tests

- Whitespace Handling test
- New File Handling test
- Rename File Handling test

PHASE 3 : Regression Tests

- Cascade effect
- Clear violation messages

Phase 4 : CI-CD

- Checkstyle compatibility
- Test filter registration in checkstyle
- Rework violations reported from CheckStyle

- XML configuration
- Doc verification

PHASE 5 : Performance tests
- Memory
- Patch processing speed
- Overhead checkstyle

NOTE : I will be testing while i fix the bugs only phase 4 and 5 will be done after implementing the new filter.

NEW FILTER IN CHECKSTYLE

PHASE 1 : Build the filter
The next step is to implement the filter class in the appropriate package, typically under `com.puppycrawl.tools.checkstyle.filters`. The class will implement the `Filter` interface and override the `accept(AuditEvent event)` method. Inside this method, we define the suppression logic. For example, in a patch-based filter, we will check whether the violation's line number exists in a set of modified lines, and suppress everything else. The implementation will be clean and avoid unnecessary complexity since it runs for every violation.

PHASE 2 : Write tests for the filter
Write unit tests proving the working of the filter . The test will contain multiple possible cases . I'll also add integration tests using Checkstyle's existing testing framework. These tests run filter against actual input files and verify the resulting violations. This involves defining expected violation messages and using helper methods like `verifyWithInlineConfigParser`. Integration tests are important because they prove that your filter works correctly within the full Checkstyle execution pipeline, not just in isolation.

PHASE 3 : Documentation

Documentation should include a clear description of what the filter does, a table of configurable properties, and at least one example showing how to use it in a Checkstyle configuration file.Next, you need to register your filter in the main documentation index so that it appears in the official Checkstyle documentation site. This typically involves updating a central XML file like `config_filters.xml`. Without this step, your filter may work technically but will not be discoverable by users, which is a common reason for PR feedback.

ONBOARDING ECLIPSE-CS

PHASE 1 : Understanding ECLIPSE-CS codebase

- Clone eclipse-cs
- Understand maven build
- Run checkstyle on eclipse -cs
- Get all the violations

PHASE 2: Applying patch-filter to eclipse-cs

- Configure checkstyle.xml in eclipse-cs
- Add patch-filters to pom.xml for testing the filter after changes.(New filter will be tested later)
- Add patch-generation
- Write test cases

PHASE 3 : Testing

- I will generate a real life patch examples like single line change , add new line etc… based on the three strategies used.
- Test the patch with the filter
- Test the patch without filter
- Compare both the results

Goal : We should lesser violations when we put filter in use ,as it suppresses the violations caused by unchanged lines.

PROJECT TIMELINE

WEEKS	START DATE	END DATE	TASKS TO ACCOMPLISH
-	May 1	May 26	Community Bonding Period
1	May 27	May 30	Fix indentationcheck(233) and rightcurltcheck(12). Rework javaDoc– PR open on rightcurly merge once dependecny bump
2	May 31	June 6	Fix CustomImportOrderCheck(55) MissingJavaDocTypecheck(22).Build pass with 0 violations. Begin fix for Improvement-1.
3	June 7	June 13	Complete Improvement-1: rewrite suppressionPatchfilter element so treewalkerchecks go through isLIneamatching(). Write unit tests covering all affected checks.
4	June 14	June 20	Improvement-2 : Fix content line bug in diffformater(2.1). Fix 2.2, 2.4,2.5. Add unit tests forgenerate Patchfiletest covering the fixes.
5	June 21	June 27	Improvement 2.3 : Add ignore whitespace boolean property. Prevent indentation-only changes. Integration tests.
6	June 28	July 4	Improvement 3 : fix file handling (3.1). Process hunks inside rename file(3.2).Tests : newfilehandlingtest,renamefilehandlingtest

7	July 5	July 11	Improvement 4 : Systematic review 150 checks and expand map in JavaPatchFilterElement .java. Support Context Strategy Checks list from empty to full
8	July 12	July 18	Improvement 5 : Add full docs as comments in patchconfig.xml. Begin work on 6.1: make neversuppressedchecks patch-aware giving clear violation messages.
9	July 19	July 25	Completer 6.1.Integration tests: cascade effect testing ,other violations or testing fails rework. Test the filter on eclipse-cs . Work and complete phase 1 and phase 2.
10	July 26	Aug 1	Eclipse-cs phase 3 : generate patch examples . Test the patches with filter on and filter off .Fix feedback. Write single line maven CI integration.
11	Aug 2	Aug 9	Run entire regression tests.Write clear documentation on the changes made. Review feedback from mentors.
12	Aug 10	Aug 18	Final week : Run complete tests, Performance tests. Final documentation and usage guides .Submit final work report to mentors .

EXPECTATIONS FROM MENTORS

This project requires implementing new modules and functionalities, in which I will need to have maintainers opinions and get help from them if I get stuck over something.
I need maintainers to take time for reviewing the new functionalities or modules and give valuable feedback so I can complete my tasks in an efficient way.

MY CONTRIBUTIONS SO FAR

S.no	Type	Link	status
1	XpathRegression test (Checkstyle)	https://github.com/checkstyle/checkstyle/pull/19357	merged
2	XpathRegression Test(Checkstyle)	https://github.com/checkstyle/checkstyle/pull/19306	merged
3	XpathRegression Test(Checkstyle)	https://github.com/checkstyle/checkstyle/pull/19267	merged
4	XpathRegression Test(Checkstyle)	https://github.com/checkstyle/checkstyle/pull/19174	merged
5	XpathRegression Test(Checkstyle)	https://github.com/checkstyle/checkstyle/pull/19163	merged
6	DOC (Patchfilter)	https://github.com/checkstyle/patch-filters/pull/372	merged
7	DOC(Patch-Filter)	https://github.com/checkstyle/patch-filters/pull/374	merged
8	Unit Test(Patch-filter)	https://github.com/checkstyle/patch-filters/pull/373	merged
9	Updated docs(Patchfilter)	https://github.com/checkstyle/patch-filters/pull/378	review
10	Fix: Right curly	https://github.com/checkstyle/patch-filters/pull/377	Review

NOTE : I know the above pr’s wont guarantee me in showing my skills, i will be sending more pr’s until the announcement.

WHY ME ?

My application is distinguished by a blend of strong foundational skills and deep, project-specific engagement.
My commitment to Checkstyle began in early March, and I have since gained a comprehensive understanding of the project’s complexity and internal workings. This hands-on experience is evident in the detailed Proposed Improvements section of this proposal.

Crucially, my work on the project is already underway. I have a proven my commitment to the project with 8 Pull Requests(will be sending more) already merged across Checkstyle and patch-filters (including XPathRegression tests, documentation, and unit tests), and 2 more currently under review. This ongoing commitment, combined with my detailed technical grasp of the problem, positions me as the ideal candidate to deliver a production-ready Patch Suppression Improvement filter.

FUTURE WORK

By contributing to checkstyle, I’ve gained precious experience of how Java works in real world projects, gained beneficial skills and knowledge of thinking broadly from different perspectives as in checkstyle a single spell difference would lead to huge errors.
I want checkstyle to be adopted in java systems across the globe , which can be achieved by sheer teamwork and support from contributors.

Continuous Integration, Mutation Testing, Code Coverage, Open-Source and community work.
Irrespective of GSoC I will continue my contribution to Checkstyle, I will make quality PRs, help newcomers and solve their doubts. I've gained important skills by contributing to Checkstyle and I wish to continue expanding my knowledge even further.

THANK YOU